

MODULE I: INTRODUCTION TO OPERATING SYSTEMS

Core Concepts, Structures, and Mechanisms



OS Concepts & Functions

OS Structures

Interrupts & System Calls

Based on textbooks by Arpaci-Dusseau, Tanenbaum, Love & Silberschatz

What is an Operating System?

Definition:

An operating system is the critical intermediary between users and computer hardware, managing resources and providing a consistent interface for application execution.



Resource Manager

Controls allocation and protection of main and secondary memory, CPU time, and storage devices.



Process Controller

Manages the lifecycle of processes, including creation, scheduling, synchronization, and termination.



File System Manager

Organizes data storage by managing file attributes, access methods, and directory structures.



Security Guardian

Enforces access control, authentication, and resource protection to prevent unauthorized access.

Core OS Functions Overview

Operating systems serve five fundamental responsibilities that ensure efficient, secure, and reliable system operation:



Process Management

Manages process lifecycle, scheduling, synchronization, and termination using algorithms like Round Robin and Priority Scheduling.



Memory Management

Controls allocation and protection of memory, implements virtual memory through paging and segmentation for efficient RAM usage.



File System Management

Organizes data storage by managing file attributes, access methods, and directory structures to ensure data integrity and accessibility.



Device Management

Interfaces with hardware through device drivers, managing I/O operations via buffering, spooling, and direct memory access (DMA).



Security & Protection

Enforces access control, authentication, and resource protection to prevent unauthorized access and maintain system integrity.

These functions work together to provide essential services that applications depend on.

Process Management

Process Lifecycle

OS manages processes through creation, execution, and termination. It handles resource allocation and deallocation during each phase.

CPU Scheduling

OS uses algorithms like Round Robin and Priority Scheduling to allocate CPU time fairly among processes.

Deadlock Handling

OS implements strategies to detect and resolve deadlocks, ensuring system responsiveness and resource utilization.

Inter-Process Communication

OS provides mechanisms like shared memory and message passing for processes to synchronize and communicate effectively.

Process States & Transitions



Memory Management

The OS controls allocation and protection of memory, implementing virtual memory to enable efficient use of RAM.

Memory Allocation

- Controls use of main memory
- Allocates memory to processes
- Protects memory from unauthorized access



Virtual Memory

- Implemented through paging
- Supports programs larger than available memory
- Enables efficient memory usage



Page Management

- Demand paging loads pages as needed
- Page replacement algorithms (e.g., LRU)
- Optimizes performance and memory usage

User Processes

Virtual Memory Address Space

Kernel Space

File System Management

The OS organizes data storage, manages file operations and directory structures, and ensures data integrity through metadata and journaling.



File Organization

- Manages file attributes and metadata
- Supports multiple access methods
- Organizes directories and pathnames



File Operations

- Provides create, read, write, delete
- Supports sequential and direct access
- Implements file sharing and locking



Directory Management

- Organizes files in hierarchical structures
- Implements pathnames and links
- Manages free space allocation



Data Integrity

- Uses metadata and checksums
- Implements journaling for crash protection
- Ensures consistent file system state

Device Management & Security

Device Management

The OS interfaces with hardware through device drivers, managing I/O operations efficiently.



Device Drivers

Software that interfaces with hardware



Buffering and Spooling

Manage data flow between devices

Security & Protection

The OS enforces access control to prevent unauthorized access.



Access Control

Manages permissions for system resources



Authentication

Verifies user identities

OS Structures Overview

Operating system architecture determines how components are organized and interact within the kernel, balancing performance, modularity, and reliability.

Simple Structure

Minimal separation between components, lacking modularity and abstraction. Simple and fast but highly unstable.

 Example: MS-DOS

Monolithic Structure

All core services run in a single kernel space. Provides high performance due to direct hardware access but sacrifices reliability.

 Example: Traditional Unix, early Linux

Micro-Kernel Structure

Only essential services run in the kernel. Other services run in user space as separate modules, enhancing stability and security.

 Example: macOS, QNX

Hybrid-Kernel Structure

Combines monolithic performance with microkernel modularity. Critical functions in kernel, non-essential services in user space.

 Example: Windows NT, macOS XNU

Layered Structure

Components organized hierarchically, with each layer using services from the layer below. Improves modularity and debugging.

 Example: UNIX, early OS designs

Modular Structure

Kernel contains core components, with additional services loaded as dynamic modules. Allows runtime extension and better maintainability.

 Example: Solaris, modern Linux

Monolithic vs Microkernel Structure



Monolithic Structure

- ☰ All core services run in a single kernel space
- ⚡ High performance due to direct hardware access
- ⚠ Reliability sacrifice - kernel bugs can crash the system
- 🔑 Traditional Unix and early Linux used this model

+ Advantages

- Simple design
- High performance
- Direct function calls

- Disadvantages

- System instability
- Security vulnerabilities
- Difficult to maintain



Microkernel Structure

- 🧩 Only essential services in kernel space; others run in user space
- 🛡 Enhanced stability and security through isolation
- ⚠ Performance overhead due to inter-module communication
- 💻 macOS and QNX use hybrid forms of this structure

+ Advantages

- System stability
- Enhanced security
- Modular design

- Disadvantages

- Performance overhead
- Complex implementation
- Message passing costs

Hybrid and Modern Structures



Hybrid Kernel

Combines monolithic performance with microkernel modularity, offering a balanced approach.

- Kernel handles critical functions directly
- Non-essential services run in user space
- Balances performance and reliability

Examples:

Windows NT, macOS XNU



Modular Structure

Kernel contains core components, with additional services loaded as dynamic modules.

- Runtime extension capability
- Improved maintainability
- Load/unload modules as needed

Examples:

Solaris, modern Linux kernels



Virtual Machines

Hypervisor abstracts hardware into isolated virtual machines, each running its own OS.

- Secure multi-tenancy
- System-level isolation
- Hardware abstraction

Examples:

Cloud computing, server virtualization

Understanding Interrupts

What are Interrupts?

Hardware or software events that cause the CPU to pause current execution and transfer control to the kernel for handling.

Hardware

- Asynchronous events
- From devices: keyboard, timer, disk
- Real-time responsiveness

Software

- Synchronous events
- System calls, exceptions
- Triggered by instruction execution

Interrupt Handling Process

- 1 Hardware/software interrupt occurs
- 2 CPU saves registers and switches to kernel mode
- 3 Uses IDT (Interrupt Descriptor Table) to find handler
- 4 Kernel executes appropriate interrupt handler

System Calls Mechanism

What are System Calls?

Functions provided by the OS to allow user programs to request privileged services (e.g., file I/O, process control).

Invocation Mechanism

Invoked via software interrupt (`int 0x80` on x86) or dedicated instructions (`syscall` on modern CPUs).

Controlled Interface

Kernel validates all inputs and enforces access control, preventing unauthorized hardware manipulation.

Performance Optimization

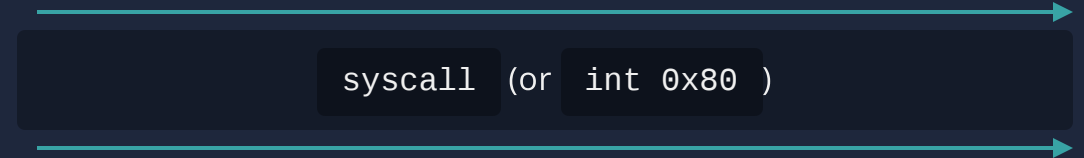
Modern CPUs provide dedicated `syscall` and `sysret` instructions for faster transitions.

System Call Flow



User Program

Request service with system call number and arguments

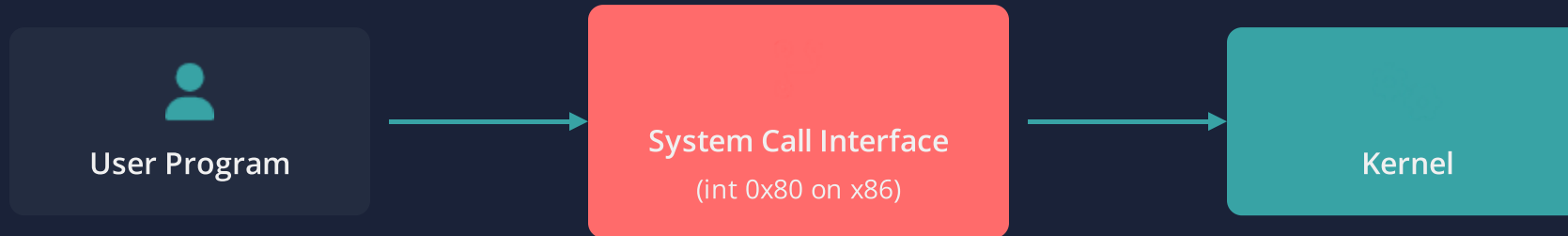


Kernel



- Save user context
- Validate inputs
- Dispatch to appropriate handler
- Execute operation
- Restore state
- Return control to user

System Call Implementation



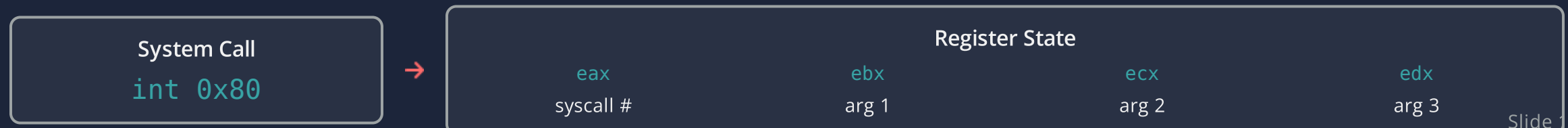
Register Usage

- > **eax**: System call number (passed as argument)
- > **ebx**, **ecx**, **edx**, etc.: System call arguments
- > **eax**: Return value after system call completion

System Call Table Dispatch

- > Kernel maintains a syscall table with pointers to handler functions
- > System call number used as index to locate appropriate handler
- > Table lookup enables efficient dispatch to correct system call handler

Example: System Call Invocation



Security and Performance Optimization



Security Enforcement

- ✓ System calls enforce a controlled interface between user programs and kernel
- ✓ Kernel validates all inputs (e.g., checking if user pointer is in valid user memory)
- ✓ Access control mechanisms prevent unauthorized hardware manipulation
- ✓ Protection rings and capability-based security maintain system integrity



Performance Optimization

- ✓ Modern CPUs provide dedicated `syscall` and `sysret` instructions
- ✓ Bypass IDT lookup for faster transitions between user and kernel mode
- ✓ Eliminate unnecessary context switching overhead
- ✓ Improve system call performance significantly



Security

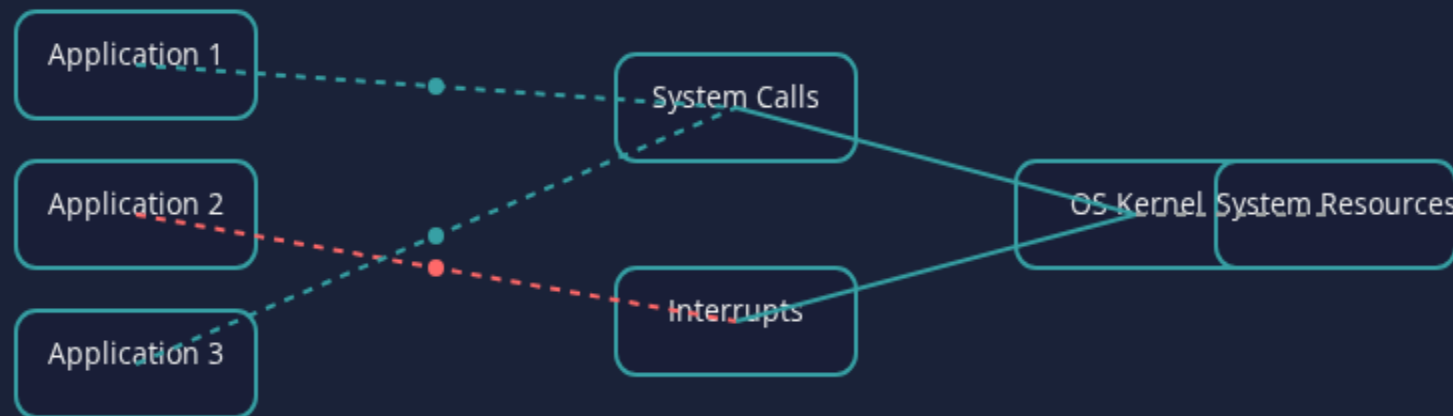
Performance



Modern operating systems balance security and performance through efficient system call implementation

Integration of OS Components

How interrupts and system calls form the essential bridge between application logic and system resources, enabling secure multitasking operation.



↔ Controlled Interface

System calls provide a controlled interface to privileged operations, preventing direct hardware manipulation.

☰ Secure Multitasking

Interrupts enable efficient context switching between processes, ensuring fair CPU allocation.

🛡 System Integrity

Together, they enforce security policies and maintain system stability.

Module Summary and Next Steps

✔ Core Principles Mastered

- Foundational understanding of OS concepts and functional components
- Architectural structures from monolithic to microkernel designs
- Low-level mechanisms including interrupts and system calls
- Resource management across process, memory, and I/O domains

➔ Next Learning Steps

-  System Programming: Implementing OS-like functionality
-  OS Security: Exploring protection mechanisms
-  Distributed Systems: Extending OS concepts to networks

"Mastery of these concepts is essential for further study in system programming, security, and distributed systems."